

Analyse statique avec Frama-C/EVA : limites, domaines relationnels, et complémentarité avec WP

TER — Master Informatique (FSI), Aix-Marseille Université

Encadrants : B. Monmege, L. Henry

June 30, 2026

Abstract

Ce rapport étudie le greffon **EVA** (*Evolved Value Analysis*) de Frama-C : son principe (interprétation abstraite), ses *réglages* (précision contre coût), et surtout ses *limites* sur les pointeurs et l'arithmétique d'adresses. On distingue les imprécisions *contournables* (jointure de branches, relations entre variables) des limites *fondamentales* (garbled mix, base faible). On montre ensuite comment le greffon **WP** (preuve déductive) prend le relais là où EVA cale, sur la base de l'article de Blanchard, Kosmatov et Louergue (HPCS 2018) et d'expériences reproductibles. *Tous les résultats EVA présentés ont été mesurés sur Frama-C 25, et les preuves WP confirmées avec Alt-Ergo 2.4.3 (tous les buts prouvés) ; codes et commandes en annexe.*

Contents

1	Introduction	2
2	Rappels : interprétation abstraite et EVA	2
2.1	Principe	2
2.2	Le <i>widening</i> : pourquoi les boucles perdent en précision	3
2.3	Le modèle mémoire : une adresse n'est pas un nombre	3
3	Régler EVA : précision contre coût	3
3.1	Le « sweet spot »	3
3.2	Le réglage dépend du programme	3
4	Domaines relationnels : les octogones	4
4.1	Non-relationnel vs relationnel	4
4.2	Expérience : « voir » la relation (vérifié)	4
5	Le garbled mix : arithmétique sur les adresses	4
5.1	Origine	4
5.2	Expérience bit à bit (vérifié)	4
5.3	Propriétés et nature	5
5.4	Un cas où EVA <i>et</i> WP butent : le pointeur tagué	5
6	Aliasing et <i>weak update</i>	5
6.1	Mise à jour forte vs faible	5
6.2	La fausse division par zéro (vérifié)	6
7	Deux familles de limites	6
8	Preuve déductive avec WP	6
8.1	L'aliasing côté WP : <code>\separated</code>	6
8.2	Boucle avec invariant : <code>reset_array</code>	7
8.3	Résultats : tout est prouvé	7

9 Complémentarité EVA / WP : la racine carrée	7
9.1 Côté EVA (mesuré)	7
9.2 Côté WP (prouvé)	8
9.3 Tableau de synthèse : « gère / ne gère pas »	8
10 E-ACSL : ce qu’aucun ne couvre seul	8
11 Étude de cas : le module AES	8
12 Conclusion	8
A Annexe : codes sources et commandes	10
A.1 01_octogones.c	10
A.2 02_bitwise_pointeur.c	10
A.3 03_alias_division.c	10
A.4 04_racine_carree.c	11
A.5 05_aes.c	11
A.6 06_pointe_vers.c	11
A.7 07_domaines.c	12
A.8 Exemples/WP/root.c (pour WP)	12

1 Introduction

La vérification de programmes C critiques (systèmes embarqués, IoT, cryptographie) repose de plus en plus sur des outils d’analyse *formelle*. Frama-C est une plateforme ouverte qui regroupe plusieurs greffons d’analyse partageant un même langage de spécification, **ACSL**. Trois greffons se complètent :

- **EVA** — calcule, par *interprétation abstraite*, une sur-approximation des valeurs possibles de chaque variable. Il prouve *automatiquement* l’absence d’erreurs à l’exécution (division par zéro, débordement, accès invalide), mais au prix de *fausses alarmes*.
- **WP** — *Weakest Precondition* : prouve la *correction fonctionnelle* vis-à-vis de contrats ACSL ; modulaire et exact, mais demande des *annotations* (préconditions, invariants de boucle).
- **E-ACSL** — traduit les annotations ACSL en code C exécutable pour vérifier *à l’exécution* ce qui n’a pas été prouvé statiquement.

Ce rapport suit le fil de l’article de référence [1], qui applique ces trois greffons à l’OS embarqué Contiki (dont son module AES). L’objectif est double : (i) comprendre *où* et *pourquoi* EVA perd en précision, et (ii) montrer la *complémentarité* EVA/WP.

Reproductibilité. Les résultats EVA ont été obtenus en commande nue (`frama-c -eva fichier.c`) sur Frama-C 25 ; les preuves WP avec `frama-c -wp -wp-rte` et Alt-Ergo 2.4.3 (tous les buts prouvés). Codes, commandes et sorties en annexe et dans **Exemples/EVA/** (mesures EVA) et **Exemples/WP/** (preuves WP).

2 Rappels : interprétation abstraite et EVA

2.1 Principe

L’*interprétation abstraite* [4] consiste à exécuter le programme non pas sur des valeurs concrètes, mais sur des *abstractions* de ensembles de valeurs. EVA utilise par défaut le domaine des **intervalles** : à chaque point de programme, il associe à chaque variable un intervalle $[a, b]$ contenant *toutes* ses valeurs possibles.

Cette approche est **correcte** (*sound*) : l'ensemble calculé contient toujours l'ensemble réel des valeurs. La contrepartie est qu'il peut être *trop grand* (sur-approximation). D'où le principe fondamental à retenir :

Une **alarme** EVA signifie « *je ne peux pas prouver que c'est sûr* », et **non** « c'est faux ». Sur du code pourtant correct, une sur-approximation peut donc produire une *fausse alarme*.

2.2 Le *widening* : pourquoi les boucles perdent en précision

Pour analyser une boucle en temps fini, EVA ne peut pas dérouler indéfiniment les itérations : il applique un opérateur d'*élargissement* (*widening*) qui saute à une sur-approximation (souvent $[0, +\infty[$). C'est rapide, mais grossier. Exemple : une boucle qui calcule $\sum_{i=0}^{99} i = 4950$ donne, par défaut, **somme** $\in [0, 2^{31}-1]$ — aucune information. On verra au §3 comment récupérer la précision.

2.3 Le modèle mémoire : une adresse n'est pas un nombre

Point crucial pour la suite : EVA représente une adresse comme un couple (**base, décalage**) — « quel objet, à quelle distance de son début » — et *non* comme un entier. **&x** est un *symbole*, pas une valeur numérique. Cette modélisation est ce qui permet de raisonner finement sur les pointeurs valides, mais c'est aussi la source du *garbled mix* (§5).

3 Régler EVA : précision contre coût

EVA expose des paramètres qui échangent de la *précision* contre du *coût* (temps, mémoire). Les trois principaux :

- **-eva-slevel** *N* — autorise EVA à propager jusqu'à *N* états *distincts* par point de programme avant de les fusionner (*state / trace partitioning*).
- **-eva-auto-loop-unroll** *N* — déroule syntaxiquement les boucles jusqu'à *N* fois.
- **-eva-widening-delay** *N* — retarde le déclenchement du widening.

3.1 Le « sweet spot »

Sur la somme $\sum_{i=0}^{99} i$, en augmentant **slevel** on observe (mesuré) :

slevel	somme
1	[0..2147483647] (rien)
10	[45..2147483647]
50	[1225..2147483647]
100	{4950} (exact)
500	{4950} (aucun gain)

La borne basse à slevel 10 est $0 + \dots + 9 = 45$; à slevel 50, c'est $0 + \dots + 49 = 1225$: **à slevel *N*, EVA suit exactement les *N* premiers tours**. La précision devient exacte exactement à slevel = 100 (le nombre d'itérations) : c'est le *sweet spot*. Au-delà, on paie sans rien gagner.

3.2 Le réglage dépend du programme

Ce seuil n'est **pas** universel : il croît avec la taille du problème. Sur la même boucle portée à 300 itérations, **slevel** 100 ne suffit plus ($[4950..2^{31}]$ — EVA n'a suivi que les 100 premiers tours), il faut **slevel** 300. On retiendra que **le coût d'EVA croît avec les données** ; ce point sera décisif au §9.

4 Domaines relationnels : les octogones

4.1 Non-relationnel vs relationnel

Le domaine des intervalles est *non relationnel* : il suit chaque variable *séparément* et **oublie les relations** entre elles. Après $y = x$, il sait $x \in [0, 100]$ et $y \in [0, 100]$, mais *pas* que $x == y$.

Le domaine des **octogones** est *relationnel* : il retient des contraintes de la forme $\pm x \pm y \leq c$. Pour $y = x$, il stocke $x - y \leq 0$ et $y - x \leq 0$, c'est-à-dire $x - y = 0$.

4.2 Expérience : « voir » la relation (vérifié)

On compare les deux domaines sur $y = x$ (code `01_octogones.c`). Le révélateur est la valeur de $x - y$:

	$(x == y)$	$x - y$
intervalles (défaut)	$\{0; 1\}$	$[-100..100]$
octogones (<code>-eva-domains octagon</code>)	$\{1\}$	$\{0\}$

Avec les intervalles, $x - y$ est totalement flou ($[-100..100]$) et l'égalité indécise ($\{0; 1\}$). Avec les octogones, $x - y = \{0\}$ *exactement* : la relation est capturée, et $(x==y)$ vaut $\{1\}$. C'est ce qui permettrait, par exemple, de prouver qu'une division par $(x - y + 1)$ est sûre.

Coût. Un domaine relationnel est plus coûteux qu'un domaine non-relationnel (il maintient des contraintes entre *paires* de variables). On l'active à la demande (`-eva-domains octagon`) quand les relations comptent.

5 Le garbled mix : arithmétique sur les adresses

5.1 Origine

Comme rappelé au §2.3, une adresse est pour EVA un couple (base, décalage), pas un nombre. Toute opération qui aurait besoin de la *valeur numérique* d'une adresse — le modulo %, les opérations *bit à bit* ($\&$, $|$, $\gg$), l'addition de deux adresses — ne peut pas être évaluée. EVA renvoie alors un **garbled mix** : une valeur abstraite opaque signifiant « quelque chose dérivé de ces bases, je ne sais plus quoi ».

5.2 Expérience bit à bit (vérifié)

Code `02_bitwise_pointeur.c` : on applique des opérations bit à bit à l'adresse de x (vue comme entier), puis la même opération à un vrai nombre.

```
Frama_C_show_each_AND_adresse :
  {{ garbled mix of &{x} (origin: Arithmetic {02_bitwise_pointeur.c:7}) }}
Frama_C_show_each_OR_adresse :
  {{ garbled mix of &{x} (origin: Arithmetic {02_bitwise_pointeur.c:8}) }}
Frama_C_show_each_SHIFT_adr :
  {{ garbled mix of &{x} (origin: Arithmetic {02_bitwise_pointeur.c:9}) }}
Frama_C_show_each_AND_nombre : [0..15]
```

Les trois opérations sur l'*adresse* ($\&$, $|$, $\gg$) produisent un garbled mix, avec l'annotation `origin: Arithmetic {ligne}` qui indique où il est né. La même opération $\& 0xF$ sur un *nombre* donne $[0..15]$, parfaitement précis.

Le point clé. Le problème n'est pas l'opération bit à bit, mais le fait de l'appliquer **à une adresse**. On le confirme au §11 : la fonction `galois_mul2` de l'AES fait $\ll$, $\gg$, $\wedge$ sur des *valeurs* et EVA reste exact.

5.3 Propriétés et nature

Le garbled mix est :

- **opaque** — aucune borne exploitable n'en sort ;
- **contaminant** — tout ce qui en dépend devient flou à son tour ;
- **irréductible** — ni une garde `if`, ni `slevel`, ni un domaine relationnel ne le récupèrent.

C'est donc une limite **fondamentale** : elle tient au *modèle mémoire*, pas au paramétrage. Aucun réglage ne la corrige.

Et les domaines dédiés ? L'Axe B du sujet évoque le domaine des *congruences* : en pratique, EVA les suit déjà nativement dans son domaine principal `cvalue` (un multiple de 4 s'affiche `[0..4000],0%4`). Il existe aussi un domaine `bitwise` pour mieux interpréter les opérateurs bit à bit. Mais sur une *adresse*, même `-eva-domains bitwise` laisse un garbled mix : la limite n'est pas l'interprétation des opérateurs, c'est le *modèle mémoire*. Vérifié, `07_domaines.c`.

5.4 Un cas où EVA et WP butent : le pointeur tagué

Contrairement à la racine carrée (§9) où WP prend le relais d'EVA, certains usages bas niveau des pointeurs mettent les *deux* greffons en échec. Cas classique, le *pointeur tagué* : on cache un bit d'information dans le bit de poids faible d'une adresse, puis on le retire pour retrouver le pointeur.

```
unsigned long tagged = (unsigned long)p | 1uL; // on marque le bit 0
int *q = (int*)(tagged & ~1uL); // q "vaut" p
return *q; // doit etre valide
```

Le programme est *correct* ($q = p$), mais le passage *adresse* $\rightarrow$ *entier* $\rightarrow$ *adresse* défait l'analyse :

- **EVA** (mesuré, `08_pointeur_tague.c`) : `tagged` et `q` deviennent des *garbled mix*, et le déréréfencement lève une **fausse alarme** `\valid_read(q)` : EVA ne peut pas garantir que le pointeur reconstruit est valide.
- **WP** (`Exemples/Comparaison/pointeur.c`) : le *cast entier* $\rightarrow$ *pointeur* sort du modèle mémoire standard (*Typed*) ; WP ne parvient pas à décharger `\valid_read(q)` ni la postcondition `\result == *p`. (Raisonnement sur les *bits* d'une adresse exigerait le modèle bas niveau « octets », bien plus lourd et non automatique.)

La cause est *commune* : une adresse n'est pas conçue pour être traitée comme un entier. C'est le pendant symétrique de la racine carrée — là WP sauvait EVA, ici *aucun* des deux ne gère. Ce genre de propriété se reporte à la vérification à l'exécution (E-ACSL) ou s'évite en réécrivant le code.

6 Aliasing et *weak update*

6.1 Mise à jour forte vs faible

Quand EVA écrit `*p = v` et qu'il connaît *exactement* la cible de `p`, il fait une *mise à jour forte* (la cible prend la valeur `v`). Mais si `p` peut pointer vers *plusieurs* variables, il fait une *mise à jour faible* (*weak update*) : il ne sait pas *laquelle* est modifiée, donc il ajoute `v` aux valeurs possibles de *chacune*, sans rien retirer.

EVA calcule l'ensemble de pointage (*points-to set*). Sur `int *p = cond ? &x : &y`, il affiche `p ∈ {&x ; &y}` : il sait que `p` vise `x` ou `y`. L'écriture `*p = 0` déclenche alors la mise à jour faible : en partant de `x = 5, y = 7`, on obtient `x ∈ {0; 5}` et `y ∈ {0; 7}` — le 0 est ajouté, les anciennes valeurs ne sont pas retirées. Vérifié, `06_pointe_vers.c`.

6.2 La fausse division par zéro (vérifié)

L'exemple canonique de l'article [1] (Fig. 1b), repris en `03_alias_division.c` :

```
if (a == 0) { x = 0; y = 5; } else { x = 5; y = 0; }
int sum = x + y;           // sum vaut TOUJOURS 5
return 10 / sum;           // EVA : fausse alarme "division par zero"
```

En *joignant* les deux branches, EVA obtient $x \in \{0, 5\}$ et $y \in \{0, 5\}$ *séparément*, donc $\text{sum} = x+y \in \{0, 5, 10\}$. Le 0 y figurant, il signale une division par zéro possible — alors que `sum` vaut *toujours* 5. EVA a perdu la *corrélation* entre `x` et `y`.

Mesuré : 1 alarme par défaut ; 0 avec `-eva-slevel 2`, qui garde les deux *traces* (les deux branches) séparées au lieu de les fusionner.

La variante *pointeur* produit la même imprécision : `int *p = cond ? &x : &y; *p = 0;` fait un *weak update* sur `x` et `y`. C'est contournable par `slevel` (le nombre d'états à garder = le nombre de cibles possibles).

7 Deux familles de limites

Le tableau ci-dessous résume la distinction centrale de ce travail :

Phénomène	Nature	Remède
Widening (boucle)	contournable	<code>slevel</code> / déroulage
Jointure de branches (§6)	contournable	<code>slevel</code>
Relation entre variables (§4)	contournable	domaine octogone
Garbled mix (§5)	fondamental	aucun (modèle mémoire)
Base faible (<code>malloc</code> en boucle)	fondamental	aucun

Savoir classer une imprécision (réglable ou fondamentale) est la compétence centrale : elle décide s'il faut *régler EVA*, *changer de domaine*, *réécrire le code*, ou *passer à WP*.

8 Preuve déductive avec WP

L où EVA *calcule* des sur-approximations, WP *prouve* qu'un programme respecte un **contrat** ACSL, en générant des obligations de preuve déchargées par des prouveurs (Alt-Ergo, CVC4). Les briques :

- **requires** / **ensures** — pré/postconditions d'une fonction ;
- **assigns** — ce que la fonction a le droit de modifier ;
- **loop invariant** / **loop variant** — propriété maintenue par une boucle / quantité qui décroît (assure la terminaison) ;
- `\separated(a,b)` — déclare que deux zones mémoire ne se chevauchent pas.

8.1 L'aliasing côté WP : `\separated`

L où EVA *calcule* les ensembles de pointage, WP *exige une spécification*. L'échange de deux entiers (Exemples/WP/swap.c) :

```
/*@ requires \valid(a) && \valid(b);
    requires \separated(a, b);
    assigns *a, *b;
    ensures *a == \old(*b) && *b == \old(*a); */
void swap(int *a, int *b){ int tmp=*a; *a=*b; *b=tmp; }
```

La précondition `\separated(a,b)` est indispensable : sans elle, si `a` et `b` étaient le même pointeur, la postcondition serait fausse. Appeler `swap(&x,&x)` fait *échouer* cette précondition — c’est ainsi que WP « gère » l’aliasing.

8.2 Boucle avec invariant : `reset_array`

```
/*@ requires 0 <= len; requires \valid(a + (0 .. len-1));
   assigns a[0 .. len-1];
   ensures \forall integer i; 0 <= i < len ==> a[i] == 0; */
void reset_array(int *a, int len){
  /*@ loop invariant 0 <= i <= len;
     loop invariant \forall integer j; 0 <= j < i ==> a[j] == 0;
     loop assigns i, a[0 .. len-1]; loop variant len - i; */
  for (int i = 0; i < len; ++i) a[i] = 0;
}
```

L’invariant exprime « tout ce qui est avant `i` est déjà à zéro » ; combiné à la condition de sortie, il *prouve* la postcondition pour *toute* taille `len`.

8.3 Résultats : tout est prouvé

Les trois fonctions du dossier `Exemples/WP/` ont été prouvées *intégralement* avec Frama-C et le prouveur Alt-Ergo 2.4.3 (`frama-c -wp -wp-rte fichier.c`) :

fichier	buts prouvés	dont Qed	dont Alt-Ergo
<code>root.c</code>	15 / 15	8	5
<code>swap.c</code>	14 / 14	9	3
<code>reset_array.c</code>	15 / 15	9	4

« Qed » regroupe les buts réglés par le simplificateur interne de WP, « Alt-Ergo » ceux déchargés par le prouveur SMT. La *terminaison* (assurée par les `loop variant`) figure parmi les buts prouvés.

9 Complémentarité EVA / WP : la racine carrée

C’est l’exemple le plus parlant de l’article (Fig. 2 et 8) ; la combinaison EVA/WP fait aussi l’objet d’un guide récent [5]. La racine carrée entière, avec une boucle *non-linéaire* :

```
int root(int N){ int R = 0; while (((R+1)*(R+1)) <= N) R = R+1; return R;
}
```

9.1 Côté EVA (mesuré)

Par défaut, EVA ne prouve pas l’absence de débordement de $(R+1)*(R+1)$: le widening donne $R \in [0, 2^{31}-1]$ et 2 alarmes d’overflow. On récupère la preuve avec `slevel`, mais le **slevel nécessaire croît avec N** :

borne sur N	slevel nécessaire
$N \leq 64$	8
$N \leq 10^6$	2000
$N \leq 10^9$ (cas de l’article)	impraticable

(Vérifié : à $N \leq 10^6$, `slevel` 8 et 100 échouent, 2000 réussit.)

9.2 Côté WP (prouvé)

Avec l'invariant `loop invariant 0 <= R*R <= N` et le variant `N - R`, WP prouve l'absence d'erreur **pour tout** $N \leq 10^9$, *indépendamment* de la taille de N . **Confirmé** : `Exemples/WP/root.c` est prouvé à **15/15** buts (Alt-Ergo 2.4.3), terminaison comprise — la complémentarité est ici démontrée directement.

La leçon. Le coût d'EVA croît avec les *données* (le *slevel* suit la taille du problème) ; WP raisonne *symboliquement* : son coût dépend de l'*effort d'annotation*, pas de N . Quand EVA cale (non-linéaire, grandes bornes), WP prend le relais. Ils sont **complémentaires**, non concurrents.

9.3 Tableau de synthèse : « gère / ne gère pas »

	EVA	WP
Annotations nécessaires	non (auto)	oui
Absence d'erreurs runtime	oui (auto)	oui (-wp-rte)
Propriétés fonctionnelles	non	oui
Boucles non-linéaires / grandes bornes	difficile	oui (invariant)
Fausse alarmes	oui	non
Vérifier une bibliothèque hors contexte	non	oui (modulaire)
Le coût croît avec...	les données	l'effort d'annotation

10 E-ACSL : ce qu'aucun ne couvre seul

Sur un logiciel réel, la preuve WP reste souvent *partielle* : on n'annote que les parties critiques. Pour le reste, **E-ACSL** traduit les annotations ACSL en C exécutable et vérifie les contrats à *l'exécution* (sur une suite de tests). L'article en donne un usage typique : une bibliothèque vérifiée par WP, appelée par une application non vérifiée, dont E-ACSL contrôle à l'exécution qu'elle respecte les préconditions. La chaîne complète est donc : **EVA** (erreurs runtime, automatique) → **WP** (fonctionnel, parties critiques) → **E-ACSL** (le reste, à l'exécution).

11 Étude de cas : le module AES

L'article applique EVA à l'AES-128 de Contiki pour prouver *automatiquement* l'absence d'erreurs à l'exécution. On reproduit l'esprit (code `05_aes.c`) :

```
static uint8_t galois_mul2(uint8_t v){ uint8_t x = (v >> 7) * 0x1b;
    return ((v << 1) ^ x); }
```

Mesuré : `galois_mul2(0x53) = {166}` (EVA est *exact* : les opérations bit à bit portent ici sur des *valeurs* `uint8_t`, pas des adresses — cf. §5). Les accès `round_keys[0][i]` pour $i \in [0, 15]$ sont valides, **0** *alarme*. Une variante boguée (boucle `i <= 16`) est **détectée** : `accessing out of bounds index. assert i < 16 (05_aes_bug.c)`. Prouver la *correction fonctionnelle* du chiffrement relèverait de WP (contrats lourds) ; pour l'objectif « absence d'erreur à l'exécution », EVA suffit et est automatique.

12 Conclusion

EVA est un outil *correct* et *automatique* pour traquer les erreurs à l'exécution, mais ses sur-approximations engendrent des fausses alarmes. On a distingué les imprécisions *contournables* — jointure de branches (*slevel*), relations entre variables (octogones), widening (déroulage) — des limites *fondamentales* liées au modèle mémoire — le garbled mix sur les adresses, la base faible. Surtout, on a montré que le coût d'EVA croît avec les *données*, tandis que WP, en raisonnant

symboliquement, prouve des propriétés que EVA ne peut atteindre (non-linéaire, grandes bornes), au prix d'annotations. EVA et WP ne sont pas concurrents : ils forment, avec E-ACSL, une chaîne de vérification complémentaire. Les expériences de ce rapport l'étayent de bout en bout : côté EVA par la mesure directe, côté WP par des preuves intégralement déchargées (Alt-Ergo 2.4.3, tous les buts prouvés).

References

- [1] A. Blanchard, N. Kosmatov, F. Loulergue. *A Lesson on Verification of IoT Software with Frama-C*. HPCS 2018. https://allan-blanchard.fr/publis/bkl_hpcs_2018.pdf
- [2] A. Blanchard. *Introduction to C program proof using Frama-C and its WP plugin*. 2017. <https://allan-blanchard.fr/publis/frama-c-wp-tutorial-en.pdf>
- [3] D. Bühler, P. Cuoq, B. Yakobowski. *EVA — The Evolved Value Analysis plug-in*. <https://www.frama-c.com/download/frama-c-eva-manual.pdf>
- [4] P. Cousot, R. Cousot. *Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints*. POPL 1977.
- [5] C. Garion, G. Hattenberger, B. Pollien, P. Roux, X. Thirioux. *A gentle introduction to C code verification using the Frama-C platform*. ISAE-SUPAERO / ONERA / ENAC, 2022. https://baptiste-pollien.fr/uploads/frama-c_verification_process.pdf

A Annexe : codes sources et commandes

Chaque programme se relance par `frama-c -eva <fichier>` ; le script `Exemples/EVA/lancer.sh` rejoue l'ensemble.

A.1 01_octogones.c

```
/* Octogones : EVA capture-t-il la relation x - y ?
 * Intervalles : frama-c -eva 01_octogones.c
 * Octogones : frama-c -eva -eva-domains octagon 01_octogones.c
 * Regarder la valeur de (x - y) et de (x == y). */
#include "__fc_builtin.h"
void main(void){
    int x = Frama_C_interval(0,100);
    int y = x; /* relation : x == y */
    int eq = (x == y);
    int dif = x - y;
    Frama_C_show_each_eq(eq); /* intervalles -> {0;1} ; octogones
    -> {1} */
    Frama_C_show_each_x_moins_y(dif); /* intervalles -> [-100..100] ;
    octogones -> {0} */
}
```

Commandes : `frama-c -eva 01_octogones.c` et `frama-c -eva -eva-domains octagon 01_octogones.c`.

A.2 02_bitwise_pointeur.c

```
/* Bitwise sur une ADRESSE -> garbled mix (vs bitwise sur un NOMBRE ->
   precis).
 * Lancer : frama-c -eva 02_bitwise_pointeur.c */
#include "__fc_builtin.h"
int x;
void main(void){
    unsigned long a = (unsigned long)&x; /* l'adresse de x vue comme un
    entier */
    Frama_C_show_each_AND_adresse(a & 0xF); /* garbled mix (origin:
    Arithmetic) */
    Frama_C_show_each_OR_adresse(a | 1); /* garbled mix */
    Frama_C_show_each_SHIFT_adr(a >> 4); /* garbled mix */
    unsigned long n = Frama_C_interval(0,255);
    Frama_C_show_each_AND_nombre(n & 0xF); /* [0..15] : precis */
}
```

A.3 03_alias_division.c

```
/* Fausse division par zero par jointure de branches (Fig 1b, Blanchard
   et al. 2018).
 * sum vaut TOUJOURS 5, mais EVA joint {0,5}x{0,5} -> sum in {0,5,10} ->
   fausse alarme.
 * Defaut : frama-c -eva 03_alias_division.c -> 1 alarme (
   fausse)
 * Corrige : frama-c -eva -eva-slevel 2 03_alias_division.c -> 0 alarme
   */
#include "__fc_builtin.h"
int f(int a){
```

```

int x, y;
if (a == 0) { x = 0; y = 5; } else { x = 5; y = 0; }
int sum = x + y;          /* toujours 5 */
return 10 / sum;          /* EVA : risque de division par zero ? */
}
void main(void){ int a = Frama_C_interval(-10,10); int r = f(a);
    Frama_C_show_each_r(r); }

```

A.4 04_racine_carree.c

```

/* Racine carree entiere (Fig 2 du papier) : boucle NON-LINEAIRE.
 * Defaut      : frama-c -eva 04_racine_carree.c      ->
    overflow non prouve
 * slevel borne : frama-c -eva -eva-slevel 8 04_racine_carree.c -> 0
    alarme (pour N<=64)
 * Mais le slevel necessaire GRANDIT avec N (8 pour 64, ~2000 pour 10^6,
    impraticable
 * pour 10^9) -> c'est la que WP prend le relais (voir wp_a_prouver/root.
    c). */
#include "__fc_builtin.h"
int root(int N){ int R = 0; while (((R+1)*(R+1)) <= N) R = R + 1; return
    R; }
void main(void){ int A = Frama_C_interval(0,64); int B = root(A);
    Frama_C_show_each_R(B); }

```

A.5 05_aes.c

```

/* AES (Contiki, simplifie) : EVA prouve l'absence d'erreur runtime, sans
    annotation.
 * galois_mul2 fait <<, >>, ^ sur des VALEURS (uint8_t) -> EVA precis.
 * Lancer : frama-c -eva 05_aes.c */
typedef unsigned char uint8_t;
static uint8_t round_keys[11][16];
static uint8_t galois_mul2(uint8_t v){ uint8_t x = (v >> 7) * 0x1b;
    return ((v << 1) ^ x); }
void main(void){
    uint8_t r = galois_mul2(0x53);          /* = 166, precis */
    Frama_C_show_each_r(r);
    for (int i = 0; i < 16; i++) round_keys[0][i] = (uint8_t)i; /* indices
        valides */
    Frama_C_show_each_cell(round_keys[0][15]);
}

```

A.6 06_pointe_vers.c

```

/* Analyse de pointeurs : EVA calcule l'ENSEMBLE DE POINTAGE (points-to
    set),
 * puis fait une mise a jour FAIBLE quand la cible est imprecise.
 * Lancer : frama-c -eva 06_pointe_vers.c */
#include "__fc_builtin.h"
int x, y;
void main(void){
    x = 5; y = 7;
    int cond = Frama_C_interval(0,1);
    int *p = cond ? &x : &y;          /* p peut viser x OU y */
}

```

```

Frama_C_show_each_p(p);          /* attendu : p dans {{ 0x ; 0y }} */
*p = 0;                          /* cible imprecise -> mise a jour FAIBLE
*/
Frama_C_show_each_x(x);          /* attendu : {0; 5} (0 ajoute, 5 non
    retire) */
Frama_C_show_each_y(y);          /* attendu : {0; 7} */
}

```

A.7 07_domaines.c

```

/* (a) Congruences : EVA les suit nativement dans cvalue (notation ,0%4).
 * (b) Domaine bitwise : -eva-domains bitwise ne recupere PAS le garbled
    mix sur adresse
 * (c'est une limite du modele memoire, pas de l'interpretation des
    operateurs bit a bit).
 * Lancer : frama-c -eva 07_domaines.c
 *          frama-c -eva -eva-domains bitwise 07_domaines.c */
#include "__fc_builtin.h"
int x;
void main(void){
    int k = Frama_C_interval(0,1000);
    int mult4 = 4 * k;                /* (a) multiple de 4 */
    Frama_C_show_each_mult4(mult4);   /* attendu : [0..4000], 0%4 */
    unsigned long a = (unsigned long)&x;
    Frama_C_show_each_adr_bitand(a & 0xF); /* (b) garbled mix, meme avec
        -eva-domains bitwise */
}

```

A.8 Exemples/WP/root.c (pour WP)

```

/* Racine carree entiere - PROUVEE par WP pour tout N <= 10^9, sans
    déroulage.
 * A comparer avec eva_experiences/04_racine_carree.c (ou EVA a besoin d'
    un slevel
 * qui grandit avec N). Source : Blanchard, Kosmatov, Loulergue, HPCS
    2018, Fig. 8.
 * Lancer : frama-c -wp -wp-rte root.c
 * Attendu : tous les buts prouves (Qed / Alt-Ergo / CVC4). */
/*@
    requires 0 <= N <= 1000000000;
    assigns \nothing;
    ensures \result * \result <= N;
    ensures N < (\result + 1) * (\result + 1);
*/
int root(int N){
    int R = 0;
    /*@
        loop invariant 0 <= R*R <= N;
        loop assigns R;
        loop variant N - R;
    */
    while (((R+1)*(R+1)) <= N) {
        R = R + 1;
    }
    return R;
}

```